

Contents

□ Trie (Prefix Tree) — Complete Interview Handbook	1
Table of Contents	1
SECTION 1 — Pattern Overview	2
SECTION 2 — Recognition Guide	3
SECTION 3 — Decision Framework	4
SECTION 4 — Why This Pattern Works	4
SECTION 5 — Algorithm Framework	5
SECTION 6 — Time & Space Complexity	6
SECTION 7 — Edge Cases	7
SECTION 8 — Pros & Cons	7
SECTION 9 — Real World Applications	8
SECTION 10 — Interview Strategy	8
SECTION 11 — Pattern Variations	9
SECTION 12 — Comparison With Other Patterns	9
SECTION 13 — Problem Classification	10
SECTION 14 — 30 Interview Problems	10
SECTION 15 — Common Mistakes	13
SECTION 16 — Optimization Techniques	13
SECTION 17 — Multiple Language Templates	14
SECTION 18 — Dry Runs	18
SECTION 19 — Advanced Concepts	18
SECTION 20 — Senior Engineer Insights	18
SECTION 21 — Cheat Sheet	19
SECTION 22 — Revision Notes (5-Minute Review)	19
SECTION 23 — Practice Roadmap	19
SECTION 24 — Memory Tricks	20
SECTION 25 — Final Summary	20

□ **Trie (Prefix Tree) — Complete Interview Handbook**

Pattern #16 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Trie section

Table of Contents

1. Pattern Overview · 2. Recognition Guide · 3. Decision Framework · 4. Why This Pattern Works · 5. Algorithm Framework · 6. Time & Space Complexity · 7. Edge Cases · 8. Pros & Cons · 9. Real World Applications · 10. Interview Strategy · 11. Pattern Variations · 12. Comparison With Other Patterns · 13. Problem Classification · 14. 30 Interview Problems · 15. Common Mistakes · 16. Optimization Techniques · 17. Multiple Language Templates · 18. Dry Runs · 19. Advanced Concepts · 20. Senior Engineer Insights · 21. Cheat Sheet · 22. Revision Notes · 23. Practice Roadmap · 24. Memory Tricks · 25. Final Summary

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

A Trie (prefix tree) is a tree structure where **each path from the root represents a prefix**, and each node represents one character position shared by every string passing through it. Words sharing a common prefix share the same path from the root, diverging only at the point their characters differ. This lets you check prefix existence, word existence, and autocomplete-style queries in $O(L)$ time (L = word/prefix length), independent of how many words are stored.

1.2 Why Was This Pattern Invented?

Storing strings in a hash set gives $O(1)$ exact-match lookup, but answering “does any word start with this prefix?” requires scanning every stored string — $O(n \cdot L)$. A Trie was invented to make prefix-based operations as fast as exact-match ones, by **physically sharing storage for common prefixes** and letting you walk directly along a prefix’s path, checking existence in $O(L)$ regardless of the total number of stored words.

1.3 Real Intuition Behind The Pattern

Imagine a **branching hallway where every room number is built letter by letter** — “CAT” and “CAR” share the same first two doors (C, A) before splitting at the third letter. Walking down the shared hallway is exactly how a Trie reuses storage and lookup time for common prefixes.

1.4 Mental Model

Each Trie node has a set of children (one per possible next character) and a boolean flag marking “a complete word ends here.” Insertion and search both walk character-by-character from the root, following (or creating) the matching child at each step.

1.5 Visual Explanation

Insert: "CAT", "CAR", "CARD", "DOG"

```
root
  C
    A
      T (end of word: CAT)
      R (end of word: CAR)
        D (end of word: CARD)
    D
      O
        G (end of word: DOG)
```

Search "CAR": root→C→A→R, R.isEndOfWord=true → FOUND

Search "CA": root→C→A, A.isEndOfWord=false → prefix exists, but not a complete word

Search "CARE": root→C→A→R→(no child 'E') → NOT FOUND

1.6 Simple Analogy

A Trie is like a **dictionary organized as a physical letter-by-letter branching path** — to find a word, you walk down one branch per letter; every word sharing your word’s prefix walks the exact same initial branches with you before splitting off.

1.7 When Should I Immediately Think About Using This Pattern?

- “Implement autocomplete/typeahead.”
- “Check if a word/prefix exists” among a large dictionary, repeatedly.
- “Longest common prefix” among a set of strings.
- “Word search in a grid with a dictionary” (Trie + backtracking combination).
- Any problem requiring **many repeated prefix-based lookups** against a fixed or growing set of strings.

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
“autocomplete,” “typeahead”	Direct signal
“prefix,” “starts with”	Direct signal
“dictionary of words,” repeated lookups	Trie signal
“word search II” (grid + dictionary)	Trie + backtracking combination
“longest common prefix”	Trie or simpler direct comparison, depending on scale
“replace words with root” (word stemming)	Trie-based shortest-prefix-match

2.2 Hidden Hints

A problem involving **many** words and **many** prefix-based queries (not just one-off) is the strongest signal — for a single query, simpler techniques suffice; Tries shine specifically when prefix queries are repeated against the same dictionary.

2.3 Interview Clues

Interviewer mentions “many queries against the same dictionary” or “as the user types” (autocomplete framing) — both imply amortizing dictionary structure setup cost across many fast queries.

2.4 Common Trick Words

“Starts with,” “prefix,” “shortest root word,” “search word with wildcards” (Trie + DFS for wildcard matching).

2.5 What Interviewers Expect

Correct Trie node design (children map + end-of-word flag), $O(L)$ complexity articulation (not $O(n)$ where n = number of words), and — for advanced problems — combining Trie with backtracking/DFS for grid-based word search.

2.6 When NOT To Use This Pattern

- Only a **single** exact-match or prefix query is needed (one-off) — building a full Trie for one query is overkill; a simple string comparison or hash set suffices.
- The **alphabet is very large or unbounded** (e.g., full Unicode) — a Trie’s per-node children structure (often an array indexed by character) becomes memory-inefficient; a hashmap-based children structure or a different structure (e.g., a suffix array) may be more appropriate.

- You need **suffix-based** (not prefix-based) queries — that's better solved with a Suffix Tree/Array, a related but distinct advanced structure.
-

SECTION 3 — Decision Framework

Do you need MANY repeated prefix-based queries (autocomplete, starts-with checks) against a dictionary?

Yes → USE A TRIE ($O(L)$ per query, shared prefix storage)

No

Is it just ONE exact-match or one-off prefix check?

Yes → Simple string comparison or hash set suffices - building a Trie is unnecessary overhead

No

Do you need SUFFIX-based queries (not prefix)?

Yes → Consider a SUFFIX TREE/ARRAY instead (different, more specialized structure)

No

Is the ALPHABET very large/unbounded (e.g., full Unicode)?

Yes → Use a HASHMAP-based children structure per node instead of a fixed-size array, or reconsider whether a Trie is the most memory-efficient choice at all

Why: A Trie's core value proposition is **amortizing shared-prefix storage and lookup cost across many queries** — for a single query, its setup cost isn't worth it; and its efficiency assumes a reasonably small, fixed alphabet (like 26 lowercase letters), degrading in memory efficiency for very large character sets.

SECTION 4 — Why This Pattern Works

Mathematical/Logical: A Trie's lookup cost for a string of length L is exactly L node traversals, **independent of how many total words n are stored** — because each traversal step only examines the single child matching the next character, never needing to compare against other stored words. This is fundamentally different from a hash set's exact-match lookup (also $O(L)$ for hashing, but no *shared partial-progress* benefit for prefix queries) or a linear scan ($O(n \cdot L)$ to check a prefix against every stored word individually).

Intuitive: Since every word sharing a prefix shares the exact same initial path in the tree, checking "does this prefix exist" only requires walking that one shared path once — you get the answer for *every* word with that prefix simultaneously, for free, just by reaching that node.

Correctness Proof: *Invariant:* after walking k characters of the query string, the current Trie node represents the state "having consumed exactly this k -character prefix, following actual inserted words." *Base case:* the root represents the empty prefix, matching any word's start. *Inductive step:* moving to the child matching the $(k+1)$ th character (if it exists) correctly extends the invariant to a $(k+1)$ -character prefix; if no matching child exists, no inserted word has this prefix, correctly concluding non-existence. *Termination:* after consuming all L characters, the current node (if reached) represents exactly the query string as a prefix; checking its `isEndOfWord`

flag correctly determines whether it's also a complete stored word, not just a prefix of a longer one. **QED.**

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework

1. Define a Trie node: a map/array of children (one per possible character) and an `isEndOfWord` boolean flag.
2. **Insert:** starting at the root, for each character in the word, move to (creating if necessary) the corresponding child; mark the final node's `isEndOfWord` = `true`.
3. **Search (exact word):** walk character by character; if any required child is missing, return `false`; at the end, return the final node's `isEndOfWord` flag.
4. **StartsWith (prefix check):** identical walk to search, but return `true` as soon as the full prefix path exists, regardless of `isEndOfWord`.

5.2 General Template

```
class TrieNode:
    children = map<char, TrieNode>()
    isEndOfWord = false

function insert(root, word):
    node = root
    for char in word:
        if char not in node.children:
            node.children[char] = new TrieNode()
        node = node.children[char]
    node.isEndOfWord = true

function search(root, word):
    node = root
    for char in word:
        if char not in node.children:
            return false
        node = node.children[char]
    return node.isEndOfWord

function startsWith(root, prefix):
    node = root
    for char in prefix:
        if char not in node.children:
            return false
        node = node.children[char]
    return true
```

5.3 Word Search II Template (Trie + Backtracking on a Grid)

```
function findWords(board, words):
    trie = buildTrie(words)
    result = set()
    for each cell (r, c) in board:
        dfs(board, r, c, trie.root, "")
    return result
```

```

function dfs(board, r, c, node, path):
    char = board[r][c]
    if char not in node.children: return
    nextNode = node.children[char]
    newPath = path + char
    if nextNode.isEndOfWord: result.add(newPath)

    mark board[r][c] as visited
    for each neighbor (nr, nc):
        if in bounds and not visited:
            dfs(board, nr, nc, nextNode, newPath)
    unmark board[r][c] as visited    # backtrack

```

5.4 Interview Thinking Process

1. "This involves many repeated prefix-based queries against a dictionary — I'll build a Trie once, then answer each query in $O(L)$."
2. "I'll design each node with a children map and an end-of-word flag, distinguishing 'this is a valid prefix' from 'this is also a complete word.'"
3. "For grid-based word search with a dictionary, I'll combine the Trie with backtracking, pruning branches the moment the current path isn't a valid prefix in the Trie."
4. "I'll state the complexity as $O(L)$ per query, independent of the number of stored words n — this is the Trie's core advantage over a hash set for prefix operations."

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
Worst Case	$O(L)$ per insert/search/prefix-check (L = string length)	$O(N \times L \times \Sigma)$ worst case, N = number of words, Σ = alphabet size (shared prefixes reduce this in practice)	Each operation walks exactly L nodes; space bounded by total characters across all inserted words (with sharing)
Average Case	$O(L)$	$O(\text{total unique prefix characters})$	Real-world dictionaries share many prefixes, reducing actual space well below the worst-case bound
Best Case	$O(L)$ (still must examine the full string)	$O(L)$ for a single inserted word	Even one word requires L node creations
Amortized	$O(L)$ per operation, independent of N	$O(\text{total characters across all words, with prefix sharing})$	The key advantage: no dependency on N for time, unlike a linear scan

Comparison: Hash set gives $O(L)$ exact-match but $O(n \cdot L)$ for prefix-existence checks (must check every stored string); Trie gives $O(L)$ for both exact-match AND prefix-existence, independent of N .

SECTION 7 — Edge Cases

Edge Case	Example	Handling
Empty string insertion	<code>insert("")</code>	The root itself becomes an end-of-word node; clarify if this is a valid input case
Empty Trie, search on any word	No words inserted yet	Search/prefix-check correctly returns false immediately (root has no children)
Single character words	"a", "i"	Handled identically to longer words — one-level-deep Trie path
Duplicate word insertion	Inserting "cat" twice	Idempotent — no special handling needed, the same path is reused and the flag remains true
Word is a prefix of another word	"car" and "card" both inserted	The "car" node must correctly have <code>isEndOfWord = true</code> even though the Trie continues deeper for "card" — a common point of confusion
Case sensitivity	"Cat" vs "cat"	Clarify whether normalization (lowercasing) is expected before insertion/search
Very long strings with shared long prefixes	URLs, file paths	Trie's shared-prefix storage is especially beneficial here, but per-node overhead can still be significant — consider a compressed trie (radix tree) if memory is tight

Common mistakes: forgetting that a word can be both a complete word AND a prefix of a longer word simultaneously (must not treat `isEndOfWord = false` as "this path doesn't exist" — the path exists as a prefix, just not as a complete word yet); not clarifying case-sensitivity/normalization requirements upfront.

SECTION 8 — Pros & Cons

Advantages: $O(L)$ prefix/word lookup independent of the number of stored words; naturally supports prefix-based operations (autocomplete, "starts with") that hash sets can't do efficiently; shared storage for common prefixes saves memory versus storing each word independently. **Disadvantages:** Per-node overhead (a children map/array for each node) can exceed a simple hash set's memory footprint when prefix sharing is minimal (e.g., mostly-distinct random strings); more complex to implement correctly than a hash set. **Trade-offs:** Trie ($O(L)$ prefix+exact queries, higher per-node memory overhead) vs. Hash Set ($O(L)$ exact-match only, $O(n \cdot L)$ for

prefix queries, simpler/lower overhead per entry) — choose Trie specifically when prefix operations are frequent. **Limitations:** Not naturally suited for suffix-based or substring (not prefix) queries; large/unbounded alphabets increase per-node memory cost significantly. **Inefficient when:** words share very few common prefixes (minimal sharing benefit) and only exact-match queries are needed — a hash set would be simpler and comparably fast.

SECTION 9 — Real World Applications

Domain	Application
Google	Search autocomplete/typeahead suggestions as you type a query
Amazon	Product search autocomplete, SKU/prefix-based catalog lookups
Meta	Username/hashtag autocomplete and prefix-based search suggestions
Mobile Keyboards (predictive text)	T9/predictive text input relies on Trie-like prefix matching for word suggestions
DNS Systems	Domain name resolution and routing table lookups (longest-prefix matching, a Trie-adjacent technique used in IP routing)
Networking	IP routing tables use a Trie variant (Patricia Trie / radix tree) for longest-prefix-match routing decisions
Spell Checkers	Efficient dictionary lookups and “did you mean” suggestions based on prefix/edit-distance combinations
IDEs/Code Editors	Autocomplete for variable/function names as you type
Bioinformatics	Trie-like structures for efficient substring/prefix matching in DNA sequence analysis
Compilers	Keyword/identifier recognition in lexical analysis sometimes uses Trie-based matching for efficiency

SECTION 10 — Interview Strategy

How seniors answer: They immediately recognize the “many repeated prefix queries” signal, state the $O(L)$ independent-of- N complexity advantage explicitly, and design the node structure (children map + end-of-word flag) clearly before coding, distinguishing “valid prefix” from “complete word” explicitly.

How juniors answer: They often reach for a hash set by default even when prefix operations dominate, missing the Trie’s specific advantage, or they build a Trie but forget the `isEndOfWord` flag, incorrectly treating any reachable node as a complete word.

Typical follow-ups: “How would you implement autocomplete showing the top- k suggestions for a prefix?” (Trie + DFS collecting all words under a prefix node, or augmenting nodes with frequency counts / a small heap). “How would you handle a very large alphabet (Unicode)?” (hashmap-based children instead of fixed-size array). “Can you combine this with a grid-based word search?” (Word Search II — Trie + backtracking).

Optimization questions: “Can you reduce memory for long chains of single-child nodes?” (discuss compressed tries / radix trees, which merge single-child chains into one edge labeled with a substring).

SECTION 11 — Pattern Variations

Variation	Description	Example
Basic Trie (Insert/Search/StartsWith)	Core operations	Implement Trie (Prefix Tree)
Trie + Backtracking (Grid Search)	Prune DFS branches using Trie prefix validity	Word Search II
Trie with Wildcard Matching	DFS branches on ‘.’ wildcard characters	Design Add and Search Words Data Structure
Trie for Longest Common Prefix	Shared-path length gives the answer directly	Longest Common Prefix (Trie-based approach)
Trie for Word Replacement (Shortest Root)	Find the shortest matching prefix among roots	Replace Words
Compressed Trie (Radix Tree)	Merges single-child chains for memory efficiency	Advanced/production-grade Trie implementations
Trie for XOR Maximization (Bitwise Trie)	Binary Trie over bit representations	Maximum XOR of Two Numbers in an Array

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
Hashing	$O(L)$ exact-match, but $O(n \cdot L)$ for prefix queries	Only exact-match/frequency queries needed, not prefix operations
Backtracking	Used together with Trie for grid-based word search, providing the pruning structure	Combine both when searching a grid against a dictionary
Suffix Tree/Array	Handles suffix/substring queries, not just prefixes	Need substring (not prefix) matching
Binary Search on Sorted Strings	$O(\log n \times L)$ prefix range queries on static sorted data	Data is static and doesn’t need insertion, and memory efficiency matters more than absolute query speed

Comparison Table

Aspect	Trie	Hash Set	Sorted Array + Binary Search
Exact-match query	$O(L)$	$O(L)$	$O(\log n \times L)$
Prefix query	$O(L)$	$O(n \times L)$	$O(\log n \times L) + \text{scan}$
Space	$O(\text{total chars, with sharing})$	$O(\text{total chars, no sharing benefit})$	$O(\text{total chars})$

Aspect	Trie	Hash Set	Sorted Array + Binary Search
Supports dynamic insertion	Yes	Yes	Costly (O(n) shift)

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	Basic insert/search/startsWith	Implement Trie (Prefix Tree), Longest Common Prefix
Medium	Wildcard search, word replacement	Design Add and Search Words Data Structure, Replace Words, Map Sum Pairs
Hard Very Hard	Trie + backtracking on grids Bitwise Tries, advanced combined structures	Word Search II Maximum XOR of Two Numbers in an Array, Stream of Characters

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Implement Trie (Prefix Tree)	Medium	Amazon, Meta, Google, Microsoft	Core insert/search/startsWith	Foundational patterns
2	Longest Common Prefix	Easy	Amazon, Meta	Trie-based or direct comparison approach	Basic prefix-sharing application
3	Design Add and Search Words Data Structure	Medium	Amazon, Meta, Google	Trie + DFS for wildcard matching	Wildcard search mastery
4	Word Search II	Hard	Amazon, Meta, Google, Microsoft	Trie + backtracking on a grid	Advanced cross-pattern combination
5	Replace Words	Medium	Amazon, Google	Trie-based shortest-prefix-match	Prefix-matching application
6	Map Sum Pairs	Medium	Google, Amazon	Trie augmented with value sums	Augmented Trie design

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
7	Maximum XOR of Two Numbers in an Array	Medium	Google, Amazon	Bitwise Trie for XOR maximization	Advanced bitwise Trie application
8	Stream of Characters	Hard	Google, Amazon	Reverse Trie for suffix-style streaming match	Advanced streaming Trie application
9	Search Suggestions System	Medium	Amazon, Meta	Trie or sorting-based prefix matching	Autocomplete-style application
10	Palindrome Pairs	Hard	Google, Amazon	Trie-based reverse-prefix matching	Advanced Trie + palindrome combination
11	Short Encoding of Words	Medium	Google, Amazon	Trie (suffix-based via reversed words) for encoding optimization	Reverse-Trie application
12	Index Pairs of a String	Medium	Google	Trie-based substring matching	Basic Trie application
13	Implement Magic Dictionary	Medium	Google, Amazon	Trie + DFS with one-character-mismatch allowance	Constrained wildcard search
14	Longest Word in Dictionary	Easy	Google, Amazon	Trie-based buildability checking	Prefix-chain validation
15	Prefix and Suffix Search	Hard	Google, Amazon	Combined prefix+suffix Trie (or combined key trick)	Advanced dual-constraint Trie
16	Concatenated Words	Hard	Google, Amazon	Trie/DP hybrid for word segmentation	Cross-pattern (Trie + DP)
17	Word Break (contrast)	Medium	Amazon, Meta	Contrast: DP-based, Trie can optimize prefix checks	Cross-pattern reinforcement
18	Word Break II (contrast)	Hard	Amazon, Meta, Google	Contrast: Backtracking + DP, Trie can optimize prefix checks	Cross-pattern reinforcement

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
19	Design Search Auto-complete System	Hard	Google, Amazon	Trie augmented with frequency + top-k retrieval	Advanced system-design-adjacent Trie
20	Count Pairs With XOR in a Range (contrast)	Hard	Google	Bitwise Trie application variant	Advanced bitwise Trie reinforcement
21	Camelcase Matching	Medium	Google	Trie-adjacent sequential matching (not a pure Trie problem)	Pattern-boundary awareness
22	Number of Matching Subsequences (contrast)	Medium	Google, Amazon	Contrast: bucket-based approach, not Trie	Pattern-boundary awareness
23	Extra Characters in a String (contrast)	Medium	Google	Contrast: DP-based with Trie-optimized dictionary lookup	Cross-pattern reinforcement
24	Sum of Prefix Scores of Strings	Medium	Google, Amazon	Trie augmented with prefix-count tracking	Augmented Trie counting
25	Delete Duplicate Folders in System	Hard	Google, Amazon	Trie-based folder-structure serialization and comparison	Advanced Trie + serialization
26	Build Array from Permutation (contrast)	Easy	Amazon	Contrast: unrelated to Trie, included for pattern-boundary practice	Pattern-boundary awareness
27	Trie-Based Autocomplete System (custom/interview variant)	Hard	Google, Amazon (system design-adjacent)	Full Trie-based product feature implementation	Applied system design

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
28	Longest Word With All Prefixes (contrast framing of #14)	Easy	Google	Reinforce prefix-chain validation	Reinforcement
29	Two Sum (contrast, hashing preferred)	Easy	Amazon	Contrast: pure hashing problem, not Trie	Pattern-boundary awareness
30	K-th Smallest in Lexicographical Order	Hard	Google, Amazon	Trie-structure-implied traversal counting (without building an explicit Trie)	Advanced implicit-Trie reasoning

SECTION 15 — Common Mistakes

1. Forgetting the `isEndOfWord` flag entirely, incorrectly treating any reachable Trie node as a complete stored word rather than just a valid prefix. *Fix*: always distinguish “prefix exists” (path exists) from “word exists” (path exists AND `isEndOfWord == true`).
2. Not handling the case where one inserted word is a prefix of another (e.g., “car” and “card” both inserted) — must correctly mark “car”’s node as `isEndOfWord = true` even though the Trie continues deeper. *Fix*: always set the flag at insertion’s final node, regardless of whether children already exist there.
3. Using a fixed-size array (e.g., size 26) for children when the alphabet is actually larger or includes uppercase/digits/Unicode. **Fix*:s clarify the character set upfront; use a hashmap for children if the alphabet isn’t small and fixed.
4. Building a full Trie for a single one-off prefix query, adding unnecessary complexity/overhead versus a simple string comparison. *Fix*: recognize when the “many repeated queries” justification for a Trie doesn’t actually apply.
5. In Word Search II (Trie + backtracking), forgetting to prune the DFS the moment the current path isn’t a valid Trie prefix, degrading to a much slower unconstrained grid search. *Fix*: always check Trie children existence before recursing into a neighbor cell.

Why people fail: the basic Trie node structure is simple, but candidates often blur “this path exists” with “this is a complete word,” a subtle but critical distinction that becomes especially important in problems with overlapping prefix/word relationships (like “car”/“card”) — getting this wrong produces plausible-looking but incorrect answers.

SECTION 16 — Optimization Techniques

- **Time:** Prune early and aggressively when combining Trie with backtracking (Word Search II) — the moment a partial path isn’t a valid Trie prefix, stop exploring that branch immediately.
- **Space:** Use a compressed Trie (radix tree) for long chains of single-child nodes to reduce per-node overhead; use a hashmap for children only when the alphabet is large or sparse, otherwise a fixed-size array is faster (better cache locality) for small fixed alphabets.

- **Readability:** Clearly separate “does this prefix path exist” logic from “is this a complete word” logic in code, mirroring the conceptual distinction.
 - **Interview performance:** Explicitly state the $O(L)$ independent-of- N advantage over a hash set for prefix queries — this is the single most important differentiator to articulate.
-

SECTION 17 — Multiple Language Templates

Java

```
class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isEndOfWord = false;
}

class Trie {
    TrieNode root = new TrieNode();
    void insert(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) node.children[idx] = new TrieNode();
            node = node.children[idx];
        }
        node.isEndOfWord = true;
    }
    boolean search(String word) {
        TrieNode node = traverse(word);
        return node != null && node.isEndOfWord;
    }
    boolean startsWith(String prefix) {
        return traverse(prefix) != null;
    }
    private TrieNode traverse(String s) {
        TrieNode node = root;
        for (char c : s.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) return null;
            node = node.children[idx];
        }
        return node;
    }
}
```

JavaScript

```
class TrieNode {
    constructor() { this.children = {}; this.isEndOfWord = false; }
}

class Trie {
    constructor() { this.root = new TrieNode(); }
    insert(word) {
        let node = this.root;
        for (const c of word) {
            if (!node.children[c]) node.children[c] = new TrieNode();
        }
        node.isEndOfWord = true;
    }
    search(word) {
        let node = this.root;
        for (const c of word) {
            if (!node.children[c]) return false;
            node = node.children[c];
        }
        return node.isEndOfWord;
    }
    startsWith(prefix) {
        let node = this.root;
        for (const c of prefix) {
            if (!node.children[c]) return false;
            node = node.children[c];
        }
        return true;
    }
}
```

```

        node = node.children[c];
    }
    node.isEndOfWord = true;
}
search(word) {
    const node = this.traverse(word);
    return node !== null && node.isEndOfWord;
}
startsWith(prefix) { return this.traverse(prefix) !== null; }
traverse(s) {
    let node = this.root;
    for (const c of s) {
        if (!node.children[c]) return null;
        node = node.children[c];
    }
    return node;
}
}

```

PHP

```

class TrieNode {
    public array $children = [];
    public bool $isEndOfWord = false;
}
class Trie {
    private TrieNode $root;
    public function __construct() { $this->root = new TrieNode(); }
    public function insert(string $word): void {
        $node = $this->root;
        foreach (str_split($word) as $c) {
            if (!isset($node->children[$c])) $node->children[$c] = new TrieNode();
            $node = $node->children[$c];
        }
        $node->isEndOfWord = true;
    }
    public function search(string $word): bool {
        $node = $this->traverse($word);
        return $node !== null && $node->isEndOfWord;
    }
    public function startsWith(string $prefix): bool { return $this->traverse($prefix) !== null; }
    private function traverse(string $s) {
        $node = $this->root;
        foreach (str_split($s) as $c) {
            if (!isset($node->children[$c])) return null;
            $node = $node->children[$c];
        }
        return $node;
    }
}

```

Python

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end_of_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for c in word:
            if c not in node.children:
                node.children[c] = TrieNode()
            node = node.children[c]
        node.is_end_of_word = True

    def _traverse(self, s):
        node = self.root
        for c in s:
            if c not in node.children:
                return None
            node = node.children[c]
        return node

    def search(self, word):
        node = self._traverse(word)
        return node is not None and node.is_end_of_word

    def starts_with(self, prefix):
        return self._traverse(prefix) is not None
```

Go

```
type TrieNode struct {
    children [26]*TrieNode
    isEnd    bool
}

type Trie struct { root *TrieNode }
func NewTrie() *Trie { return &Trie{root: &TrieNode{}} }
func (t *Trie) Insert(word string) {
    node := t.root
    for _, c := range word {
        idx := c - 'a'
        if node.children[idx] == nil {
            node.children[idx] = &TrieNode{}
        }
        node = node.children[idx]
    }
    node.isEnd = true
}
func (t *Trie) traverse(s string) *TrieNode {
    node := t.root
```



```

    for _, c := range s {
        idx := c - 'a'
        if node.children[idx] == nil {
            return nil
        }
        node = node.children[idx]
    }
    return node
}

func (t *Trie) Search(word string) bool {
    node := t.traverse(word)
    return node != nil && node.isEnd
}

func (t *Trie) StartsWith(prefix string) bool { return t.traverse(prefix) != nil }

```

C++

```

struct TrieNode {
    TrieNode* children[26] = {};
    bool isEndOfWord = false;
};

class Trie {
    TrieNode* root;
public:
    Trie() { root = new TrieNode(); }
    void insert(string word) {
        TrieNode* node = root;
        for (char c : word) {
            int idx = c - 'a';
            if (!node->children[idx]) node->children[idx] = new TrieNode();
            node = node->children[idx];
        }
        node->isEndOfWord = true;
    }
    TrieNode* traverse(string s) {
        TrieNode* node = root;
        for (char c : s) {
            int idx = c - 'a';
            if (!node->children[idx]) return nullptr;
            node = node->children[idx];
        }
        return node;
    }
    bool search(string word) {
        TrieNode* node = traverse(word);
        return node != nullptr && node->isEndOfWord;
    }
    bool startsWith(string prefix) { return traverse(prefix) != nullptr; }
};

```

SECTION 18 — Dry Runs

Small Input

Insert "cat", "car", "card"; then search "car", "ca", "care"

```
insert("cat"): root→c→a→t, t.isEndOfWord=true
```

```
insert("car"): root→c→a→r (new), r.isEndOfWord=true
```

```
insert("card"): root→c→a→r→d (new), d.isEndOfWord=true
```

```
search("car"): root→c→a→r exists, r.isEndOfWord=true → return TRUE
```

```
search("ca"): root→c→a exists, a.isEndOfWord=false → return FALSE (valid prefix, not a word)
```

```
search("care"): root→c→a→r exists, but no child 'e' from r → return FALSE
```

```
startsWith("ca"): root→c→a exists → return TRUE
```

Large Input (Conceptual)

Inserting 100,000 words with an average length of 10 characters costs at most 1,000,000 node creations (fewer with prefix sharing); each subsequent search/prefix-check costs $O(10)$, completely independent of the 100,000 stored words — a dramatic advantage over a hash-set-based $O(100,000 \times 10)$ prefix scan.

Corner Case

Empty Trie, `search("cat")`: root has no children → traverse returns null immediately at the first character → correctly returns false. Insert "" (empty string): the root itself gets `isEndOfWord = true` — `search("")` would then correctly return true if this edge case is supported by the problem.

SECTION 19 — Advanced Concepts

- **Bitwise Trie for XOR maximization:** representing numbers as fixed-length binary strings and inserting them into a Trie over bits (0/1 children) allows finding the number that maximizes XOR with a query number in $O(32)$ (or $O(\text{bit-length})$) by greedily choosing the *opposite* bit at each level when possible — a powerful, non-obvious application of the Trie concept beyond string prefixes.
 - **Compressed Trie (Radix Tree / Patricia Trie):** merges chains of single-child nodes into a single edge labeled with a substring, dramatically reducing node count and memory for datasets with many long, uniquely-diverging suffixes after a shared prefix (heavily used in IP routing tables).
 - **Trie + DP hybrid (Concatenated Words, Word Break):** using a Trie to make the “is this substring a dictionary word” check $O(L)$ instead of $O(L)$ hash lookups repeated across many overlapping substrings can meaningfully speed up DP solutions that repeatedly query the same dictionary.
 - **Suffix-based Tries (reversed-word insertion):** for problems needing suffix matching (Short Encoding of Words, Stream of Characters read in reverse), inserting reversed words into a Trie transforms a suffix problem into a prefix problem, directly reusing all standard Trie techniques.
-

SECTION 20 — Senior Engineer Insights

Staff engineers recognize the Trie as the canonical data structure for **any repeated prefix-based query workload** — search autocomplete, IP routing (longest-prefix match), and predictive text all reduce to the same underlying structure. They also recognize its practical limitation (per-node memory overhead for sparse/large alphabets) and know when to reach for a compressed

variant (radix tree) in memory-constrained production systems. Interviewers evaluate whether a candidate can generalize beyond “Trie = string dictionary” to recognize non-obvious applications (bitwise Tries for XOR problems, reversed-word Tries for suffix problems) — this generalization is what separates deep pattern understanding from narrow memorization of “the Trie problem.”

SECTION 21 — Cheat Sheet

PATTERN: Trie (Prefix Tree)

RECOGNIZE: "autocomplete," "prefix," "starts with," many repeated queries against a dictionary

TEMPLATE:

TrieNode: children map/array + isEndOfWord flag

insert(word): walk/create path character by character, mark final node isEndOfWord=true

search(word): walk path; if any char missing, false; else return final node's isEndOfWord

startsWith(prefix): walk path; return true if the full path exists (regardless of isEndOfWord)

COMPLEXITY: $O(L)$ per operation, independent of N (number of stored words)

KEY PROOF: shared-prefix paths mean one traversal answers "does this prefix exist" for ALL words sharing

WATCH FOR: isEndOfWord vs path-exists distinction, alphabet size assumptions, one-off query overkill

DOESN'T APPLY WHEN: single one-off query (use simple comparison), suffix/substring queries (use Suffix Tr

SECTION 22 — Revision Notes (5-Minute Review)

- Trie node = children map/array + isEndOfWord flag.
 - $O(L)$ per insert/search/prefix-check, independent of the number of stored words N .
 - “Path exists” (valid prefix) $\neq$ “isEndOfWord=true” (complete word) — always distinguish.
 - Combine with Backtracking for grid-based word search (Word Search II) — prune the moment the path isn’t a valid Trie prefix.
 - Bitwise Trie generalizes the concept to XOR-maximization problems.
 - Compressed Trie (radix tree) reduces memory for long single-child chains.
-

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Core Trie operations	Implement Trie (Prefix Tree) (208), Longest Common Prefix (14)
Intermediate	Wildcard search, word replacement	Design Add and Search Words Data Structure (211), Replace Words (648)
Advanced	Grid-based combination, augmented Tries	Word Search II (212), Map Sum Pairs (677)
Expert	Bitwise Tries, advanced hybrid structures	Maximum XOR of Two Numbers in an Array (421), Palindrome Pairs (336), Stream of Characters (1032)

SECTION 24 — Memory Tricks

- **Mnemonic:** “Shared Paths, Flagged Ends” (SPFE) — Shared prefix Paths, Flagged word Ends.
 - **Visualization:** A **branching hallway where rooms are spelled out letter by letter**, and shared prefixes share the same hallway.
 - **Recognition shortcut:** “Autocomplete/prefix/starts-with” + many repeated queries against the same dictionary → Trie, immediately.
-

SECTION 25 — Final Summary

A Trie exploits shared string prefixes by storing them once as a shared tree path, enabling $O(L)$ prefix and exact-match queries completely independent of how many words are stored — a decisive advantage over hash sets for prefix-heavy workloads like autocomplete. The single most important thing to remember forever: **a Trie node reaching the end of your query string means the prefix exists, but you must separately check the `isEndOfWord` flag to know if it’s also a complete stored word — conflating these two facts is the most common and consequential Trie bug.** The concept generalizes powerfully beyond strings: any fixed-length, small-alphabet sequence (like the binary digits of a number) can be indexed the same way, as seen in bitwise Tries for XOR-maximization problems.